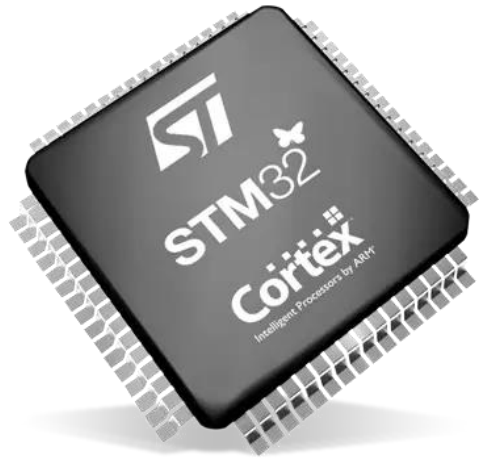




**STM32**    
**CubeIDE**

# ADC, DMA, STM32F4

---

# Agenda

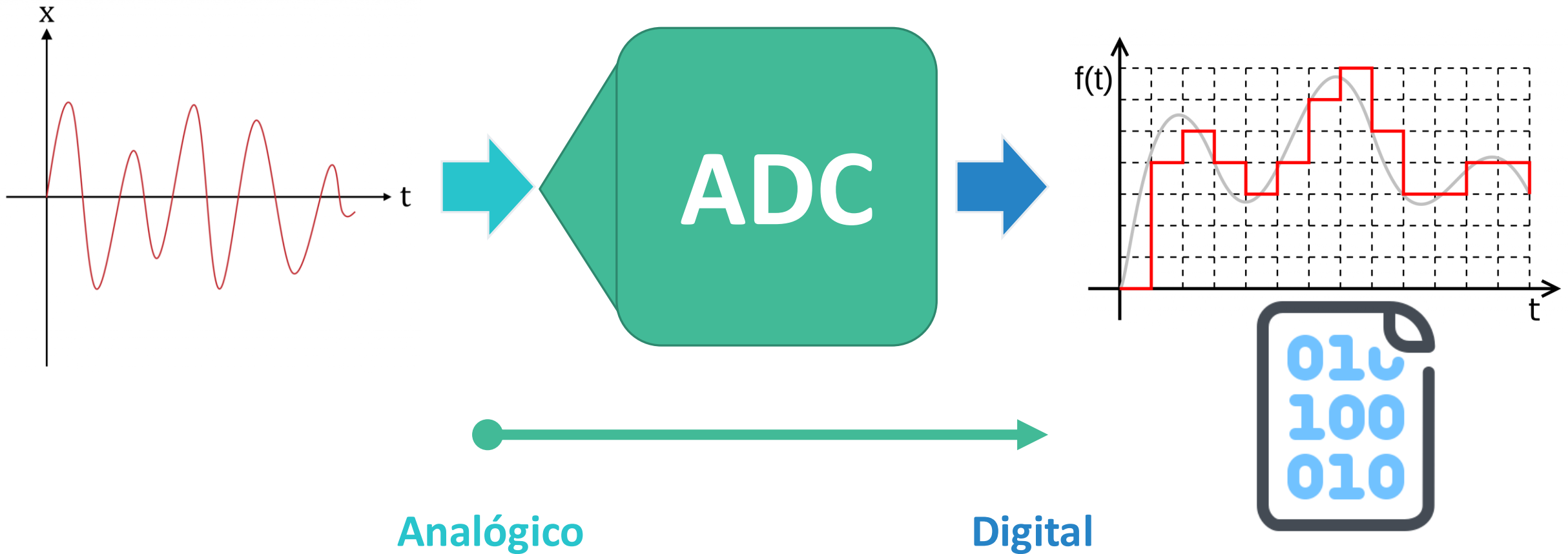
---

- ADC
- DMA

# Módulo ADC

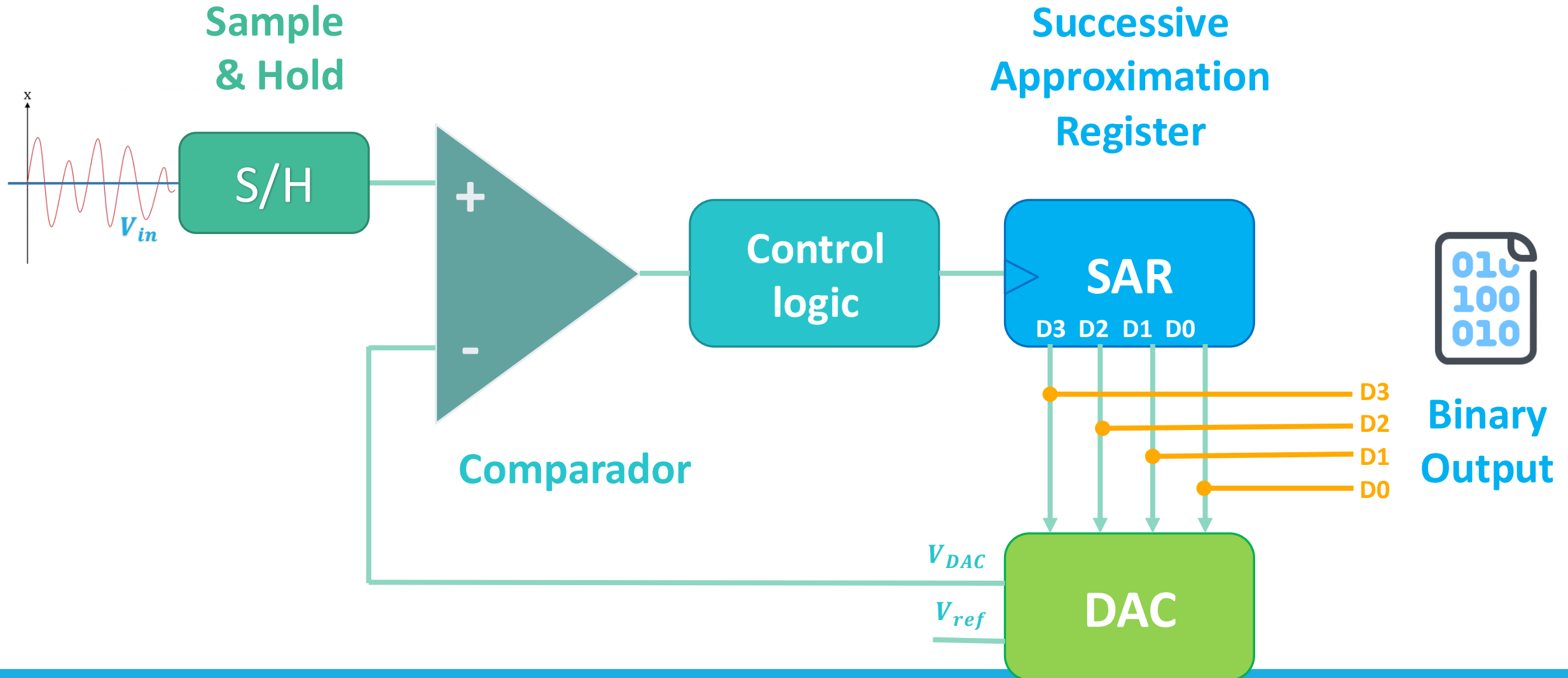
---

# Módulo ADC



# Módulo ADC SAR

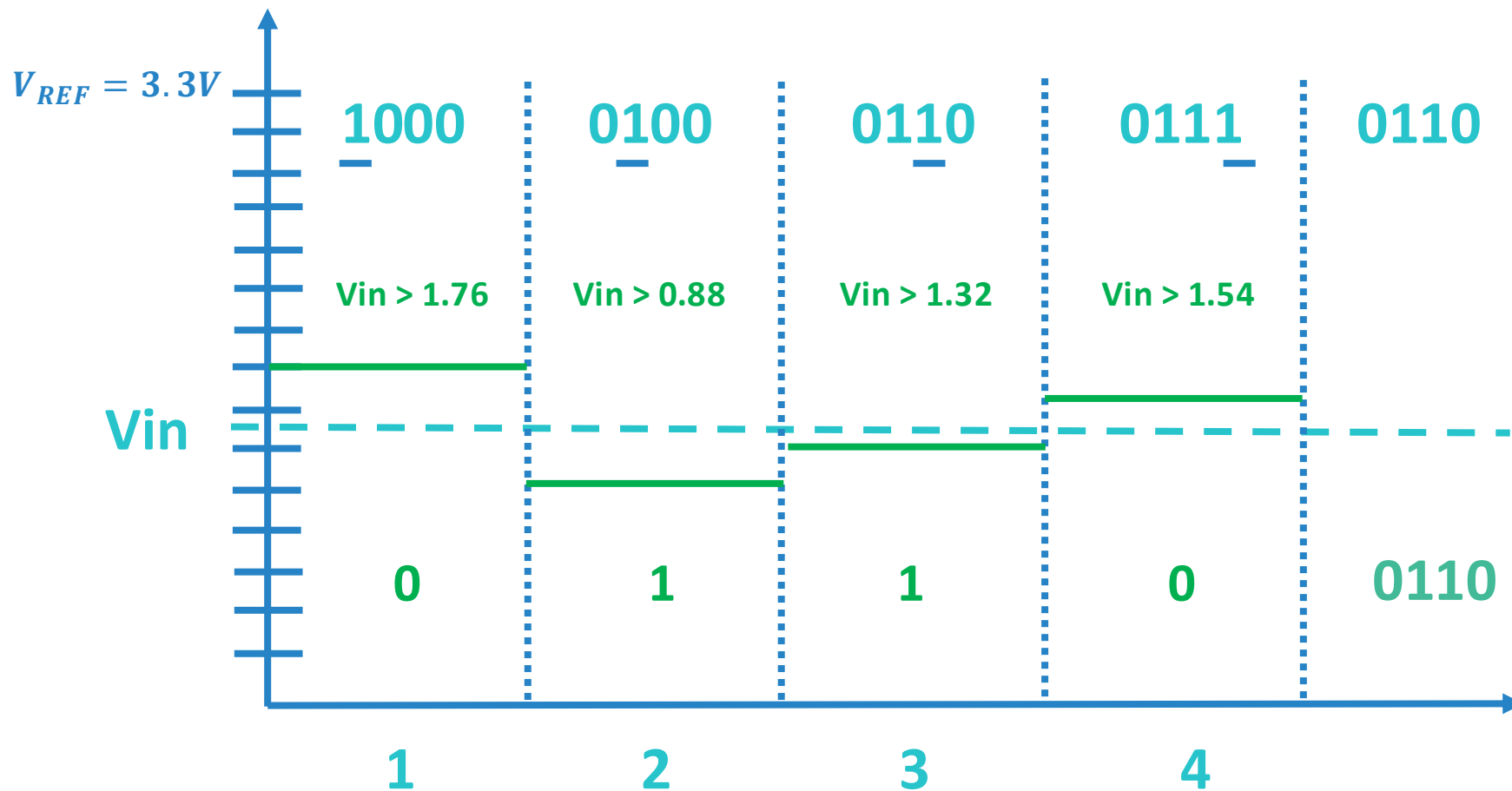
El ADC del STM32F4 es de 12 bits tipo SAR, por lo tanto cada conversión sigue este principio



# Módulo ADC SAR

$$V_{in} = 1.4V \quad V_{REF} = 3.3V$$

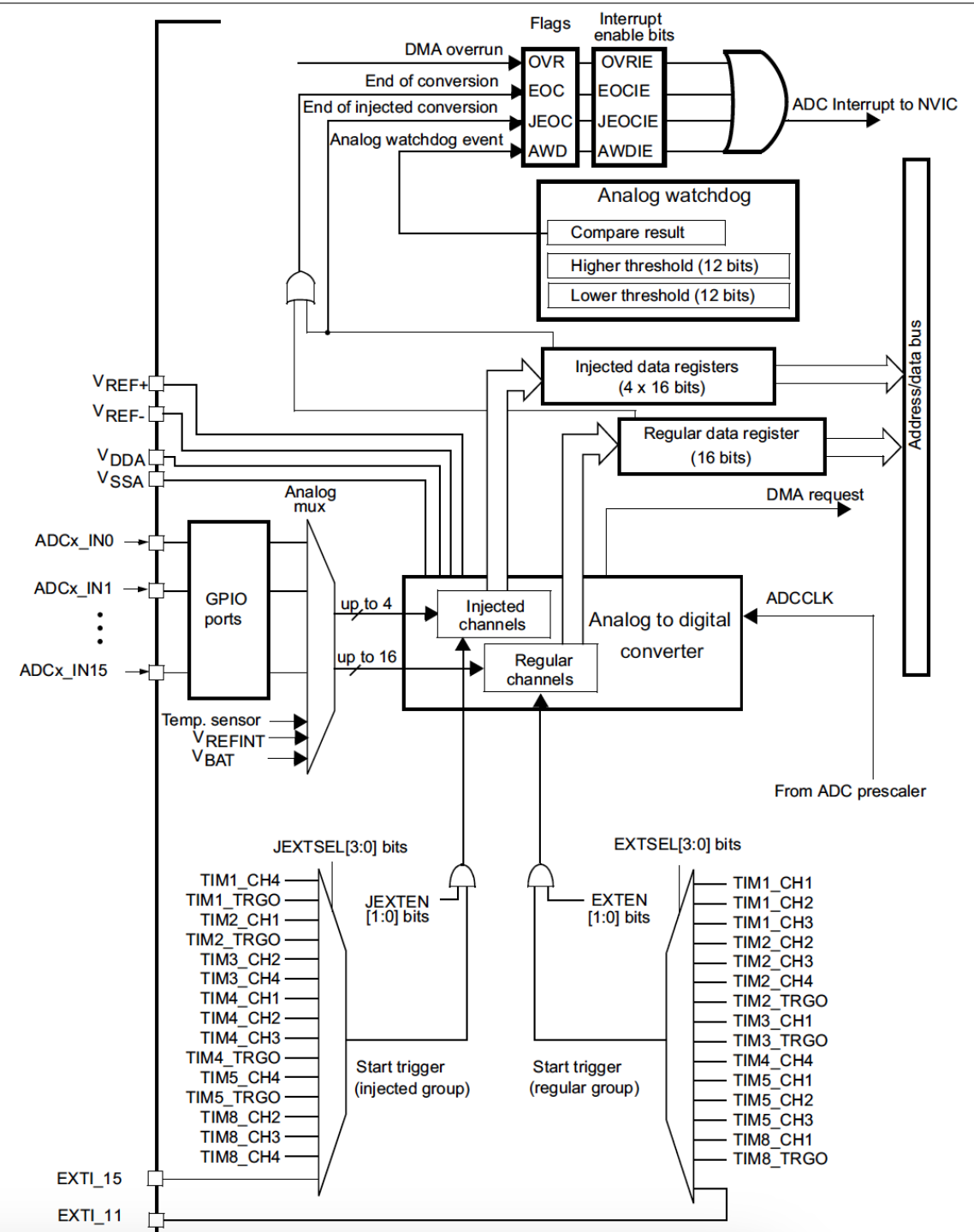
4-bit ADC resolución 0.22V



| ADC         | Voltaje     |
|-------------|-------------|
| 1111        | 3.3         |
| 1110        | 3.08        |
| 1101        | 2.86        |
| 1100        | 2.64        |
| 1011        | 2.42        |
| 1010        | 2.2         |
| 1001        | 1.98        |
| <b>1000</b> | <b>1.76</b> |
| <b>0111</b> | <b>1.54</b> |
| <b>0110</b> | <b>1.32</b> |
| 0101        | 1.1         |
| <b>0100</b> | <b>0.88</b> |
| 0011        | 0.66        |
| 0010        | 0.44        |
| 0001        | 0.22        |
| 0000        | 0           |

# Módulo ADC

- 3 módulos de hasta 24 canales seleccionables
- 2.4 MSPS
- Resolución configurable de hasta 12-bits
- Interrupciones
- Modos de conversión uno solo o continuo
- Modo escaneo
- Alineación de datos
- Tiempo de muestreo programable
- Fuentes de disparo externo configurable
- Modo discontinuo
- DMA
- Voltaje 2.4V a 3.6V a full speed y 1.8V a baja velocidad
- Rango ADC:  $V_{REF-} \leq V_{IN} \leq V_{REF+}$



$$T_{\text{conv}} = \frac{\text{Sampling Cycles} + \text{Conversion Cycles}}{\text{ADC CLOCK}}$$

**SMPx[2:0]:** Channel x sampling time selection

These bits are written by software to select the sampling time individually for each channel. During sampling cycles, the channel selection bits must remain unchanged.

Note: 000: 3 cycles  
 001: 15 cycles  
 010: 28 cycles  
 011: 56 cycles  
 100: 84 cycles  
 101: 112 cycles  
 110: 144 cycles  
 111: 480 cycles

### Fast conversion mode

It is possible to perform faster conversion by reducing the ADC resolution. The RES bits are used to select the number of bits available in the data register. The minimum conversion time for each resolution is then as follows:

- 12 bits: 3 + 12 = 15 ADCCLK cycles
- 10 bits: 3 + 10 = 13 ADCCLK cycles
- 8 bits: 3 + 8 = 11 ADCCLK cycles
- 6 bits: 3 + 6 = 9 ADCCLK cycles

# Tiempo de conversión

- **Sampling Cycles:** está en términos de ciclos de ADC, es el tiempo que le toma a un canal particula realizar una muestra de datos
- **Conversion Cycles:** está en términos de ciclos de ADC, es el tiempo tomado por el ADC en convertir la data muestreada.
- **ADC Clock:** el reloj con el que el ADC está corriendo, en este caso sería el reloj **APB2**



# Tiempos de conversión

---

| Resolución | Tiempo conversión | Aplicación típica |
|------------|-------------------|-------------------|
| 12 bits    | Más lento         | Sensores          |
| 8 bits     | Más rápido        | Audio             |



$$T_{conv} = \frac{\text{Sampling Cycles} + \text{Conversion Cycles}}{\text{ADC CLOCK}}$$

**SMPx[2:0]:** Channel x sampling time selection

These bits are written by software to select the sampling time individually for each channel.  
During sampling cycles, the channel selection bits must remain unchanged.

Note: 000: 3 cycles  
 001: 15 cycles  
 010: 28 cycles  
 011: 56 cycles  
 100: 84 cycles  
 101: 112 cycles  
 110: 144 cycles  
 111: 480 cycles

### Fast conversion mode

It is possible to perform faster conversion by reducing the ADC resolution. The RES bits are used to select the number of bits available in the data register. The minimum conversion time for each resolution is then as follows:

- 12 bits: 3 + 12 = 15 ADCCLK cycles
- 10 bits: 3 + 10 = 13 ADCCLK cycles
- 8 bits: 3 + 8 = 11 ADCCLK cycles
- 6 bits: 3 + 6 = 9 ADCCLK cycles

## Ejemplo

APB2 = 16 MHz

Prescaler de 8

ADC Clock: 2 MHz

$$T_{conv} = \frac{480 + 15}{2,000,000Hz} = 247.5 \mu Seg$$

Ojo: El ADC no corre directamente a 16 MHz. Se divide por el prescaler.

# Selección de canales

Los módulos ADC cuentan con multiplexores que le permiten obtener la señal de diferentes canales o entradas.

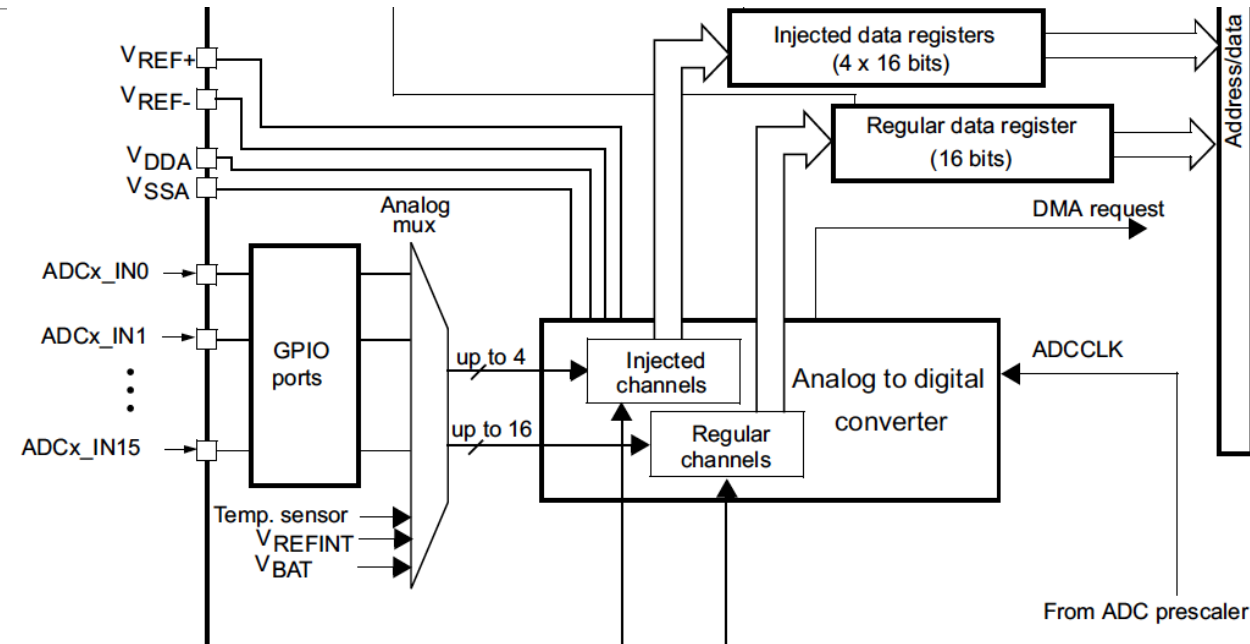
Los canales pueden compartirse entre conversores.

Se organizan en dos grupos:

- Regular Group
- Injected Group

Cada grupo puede llegar a tener hasta 16.

Los canales pueden ser convertidos en una misma operación de forma secuencial, esto es configurable.



Ejemplos:

Regular: lectura continua de sensor

Injected: lectura de emergencia (ej: corriente en sobrecarga)

# Grupos

---

## Regular Group

- Puede conformar por hasta 16 miembros

## Injected Group

- Puede estar formado por hasta 4 miembros

El orden en que se incluyen los canales define el orden de la secuencia de conversión

Es posible repetir canales dentro de los miembros

# Modos de conversión

---

## Modo Single conversión:

- ADC realiza una única conversión del canal seleccionado cada vez que se inicia
- El inicio puede ser por software o por trigger externo
- Una vez realizada la conversión, el adc detiene el muestreo

## Modo Continuo:

- ADC realiza una conversión tras otra, al terminar una conversión inicia la siguiente de forma automática
- El inicio puede ser por software o por trigger externo

## Modo Scan (multicanal):

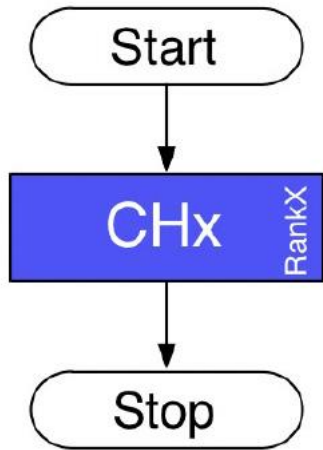
- Utilizado para muestrear un grupo de canales
- Se realiza una conversión a cada canal del grupo en el orden configurado

## Modo Discontinuo:

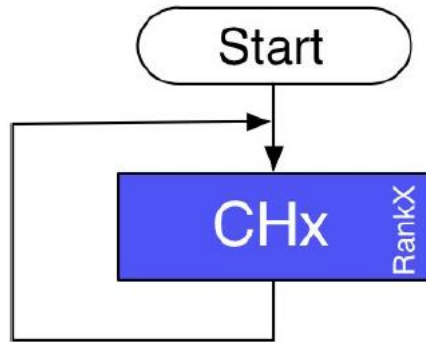
- Realiza conversiones parciales del grupo configurado. (8 canales por operación)

Cuando usamos modo continuo sin DMA, la CPU debe leer cada conversión. Aquí es donde DMA se vuelve fundamental.

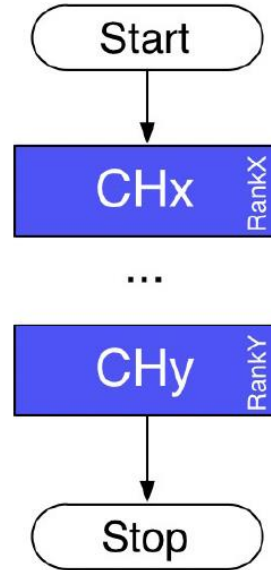
# Modos de conversión



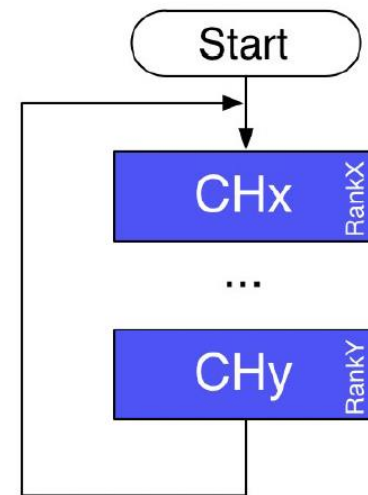
**Single  
conversion**



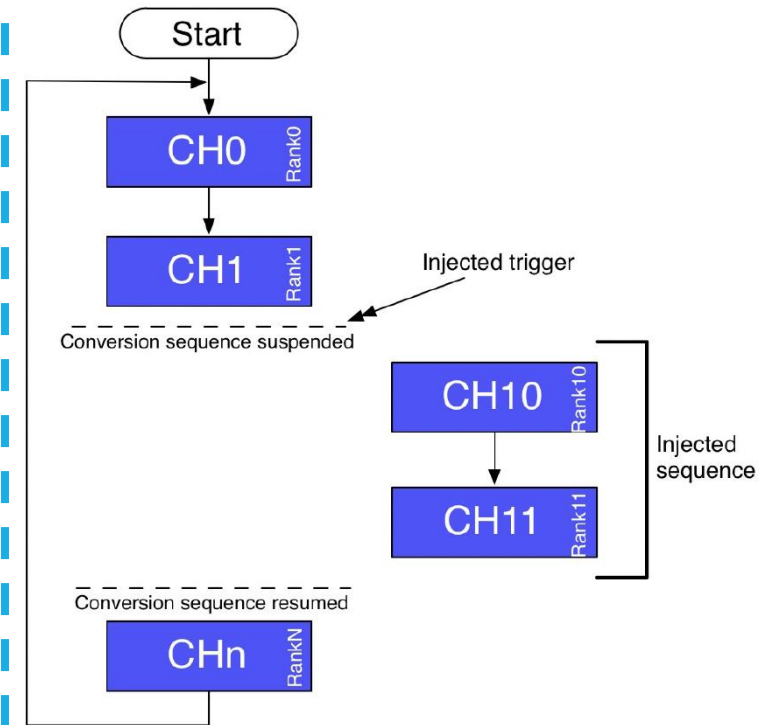
**Single  
continous**



**Scan: multi-  
channels**



**Scan:  
continous**



**Injected**

# Triggers

**Table 87. External trigger for regular channels**

| Source          | Type                                | EXTSEL[3:0] |
|-----------------|-------------------------------------|-------------|
| TIM1_CH1 event  | Internal signal from on-chip timers | 0000        |
| TIM1_CH2 event  |                                     | 0001        |
| TIM1_CH3 event  |                                     | 0010        |
| TIM2_CH2 event  |                                     | 0011        |
| TIM2_CH3 event  |                                     | 0100        |
| TIM2_CH4 event  |                                     | 0101        |
| TIM2_TRGO event |                                     | 0110        |
| TIM3_CH1 event  |                                     | 0111        |
| TIM3_TRGO event |                                     | 1000        |
| TIM4_CH4 event  |                                     | 1001        |
| TIM5_CH1 event  |                                     | 1010        |
| TIM5_CH2 event  |                                     | 1011        |
| TIM5_CH3 event  |                                     | 1100        |
| TIM8_CH1 event  |                                     | 1101        |
| TIM8_TRGO event |                                     | 1110        |
| EXTI line11     | External pin                        | 1111        |

**Table 88. External trigger for injected channels**

| Source          | Connection type                     | JEXTSEL[3:0] |
|-----------------|-------------------------------------|--------------|
| TIM1_CH4 event  | Internal signal from on-chip timers | 0000         |
| TIM1_TRGO event |                                     | 0001         |
| TIM2_CH1 event  |                                     | 0010         |
| TIM2_TRGO event |                                     | 0011         |
| TIM3_CH2 event  |                                     | 0100         |
| TIM3_CH4 event  |                                     | 0101         |
| TIM4_CH1 event  |                                     | 0110         |
| TIM4_CH2 event  |                                     | 0111         |
| TIM4_CH3 event  |                                     | 1000         |
| TIM4_TRGO event |                                     | 1001         |
| TIM5_CH4 event  |                                     | 1010         |
| TIM5_TRGO event |                                     | 1011         |
| TIM8_CH2 event  |                                     | 1100         |
| TIM8_CH3 event  |                                     | 1101         |
| TIM8_CH4 event  |                                     | 1110         |
| EXTI line15     | External pin                        | 1111         |



# Estructuras de configuración ADC

---

```
typedef struct {
    ADC_TypeDef      *Instance;           /* Pointer to ADC descriptor */
    ADC_InitTypeDef  Init;               /* ADC initialization parameters */
    __IO uint32_t    NbrOfCurrentConversionRank; /* ADC number of current conversion rank */
    DMA_HandleTypeDef *DMA_Handle;       /* Pointer to the DMA Handler */
    HAL_LockTypeDef  Lock;               /* ADC locking object */
    __IO uint32_t    State;              /* ADC communication state */
    __IO uint32_t    ErrorCode;          /* Error code */
} ADC_HandleTypeDef;
```

```

typedef struct {
    uint32_t ClockPrescaler;      /* Selects the ADC clock frequency */
    uint32_t Resolution;          /* Configures the ADC resolution mode */
    uint32_t ScanConvMode;        /* The scan sequence direction. */
    uint32_t ContinuousConvMode; /* Specifies whether the conversion is performed in
                                   Continuous or Single mode */

    uint32_t DataAlign;           /* Specifies whether the ADC data alignment
                                   is left or right */

    uint32_t NbrOfConversion;     /* Specifies the number of ranks that will be converted
                                   within the regular group sequencer */

    uint32_t NbrOfDiscConversion; /* Specifies the number of discontinuous conversions in
                                   which the main sequence of regular group */

    uint32_t DiscontinuousConvMode; /* Specifies whether the conversion sequence of regular
                                   group is performed in Complete-sequence/Discontinuous
                                   sequence */

    uint32_t ExternalTrigConv;     /* Select the external event used to trigger the start
                                   of conversion */

    uint32_t ExternalTrigConvEdge; /* Select the external trigger edge and enable it */
    uint32_t DMAContinuousRequests; /* Specifies whether the DMA requests are performed in
                                   one shot or in continuous mode */

    uint32_t EOCSelction;          /* Specifies what EOC (End Of Conversion) flag is used
                                   for conversion polling and interruption */
} ADC_InitTypeDef;

```

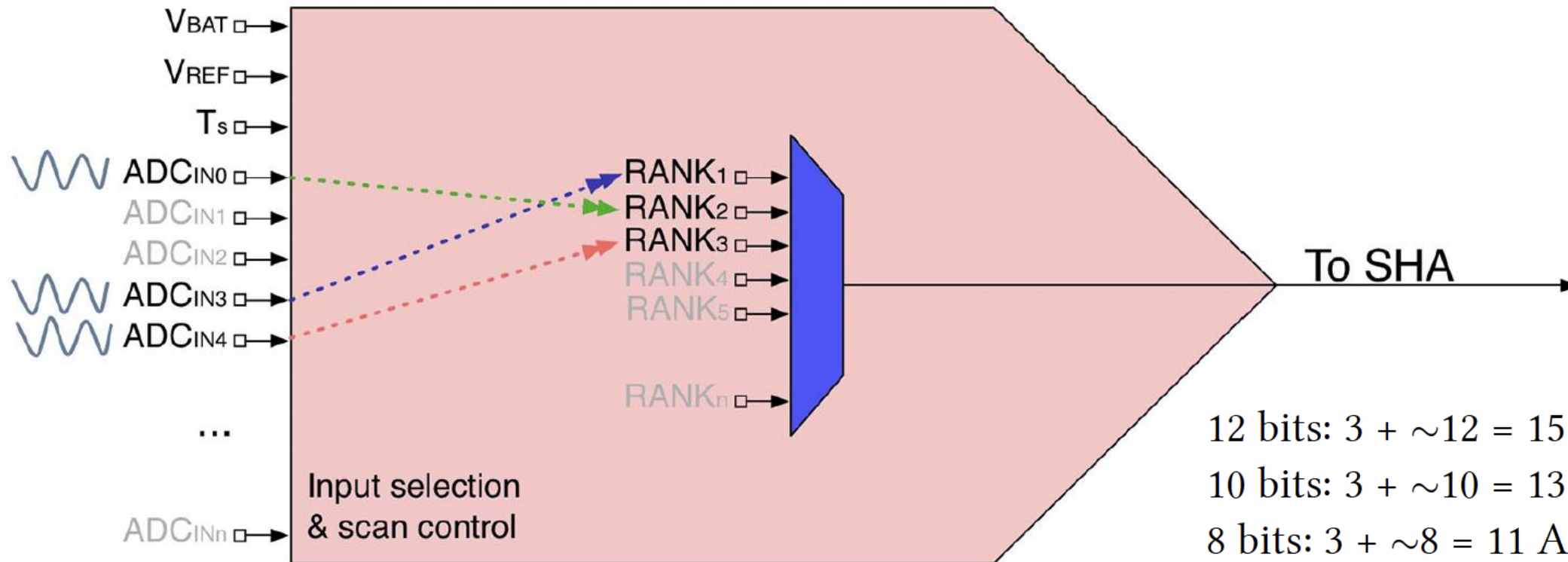
---

| ADC resolution     | Description           |
|--------------------|-----------------------|
| ADC_RESOLUTION_12B | ADC 12-bit resolution |
| ADC_RESOLUTION_10B | ADC 10-bit resolution |
| ADC_RESOLUTION_8B  | ADC 8-bit resolution  |
| ADC_RESOLUTION_6B  | ADC 6-bit resolution  |

# Selección del canal

```
typedef struct {  
    uint32_t Channel;  
    uint32_t Rank;  
    uint32_t SamplingTime;  
    uint32_t Offset;  
} ADC_ChannelConfTypeDef;
```

```
/* Specifies the channel to configure into ADC rank */  
/* Specifies the rank ID */  
/* Sampling time value for the selected channel */  
/* Reserved for future use, can be set to 0 */
```



12 bits:  $3 + \sim 12 = 15$  ADCCLK cycles  
10 bits:  $3 + \sim 10 = 13$  ADCCLK cycles  
8 bits:  $3 + \sim 8 = 11$  ADCCLK cycles  
6 bits:  $3 + \sim 6 = 9$  ADCCLK cycles

# Instrucciones HAL ADC

---

Función con polling para iniciar secuencia de ADC

```
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout);
```

Función para obtener el valor de ADC

```
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc);
```

Ejemplo:

```
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
```

```
rawValue = HAL_ADC_GetValue(&hadc1);
```

```
temp = ((float)rawValue) / 4095 * 3300;
```

```
temp = ((temp - 760.0) / 2.5) + 25;
```

# Instrucciones HAL ADC

---

Función con Interrupción para iniciar secuencia de ADC

```
HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc);
```

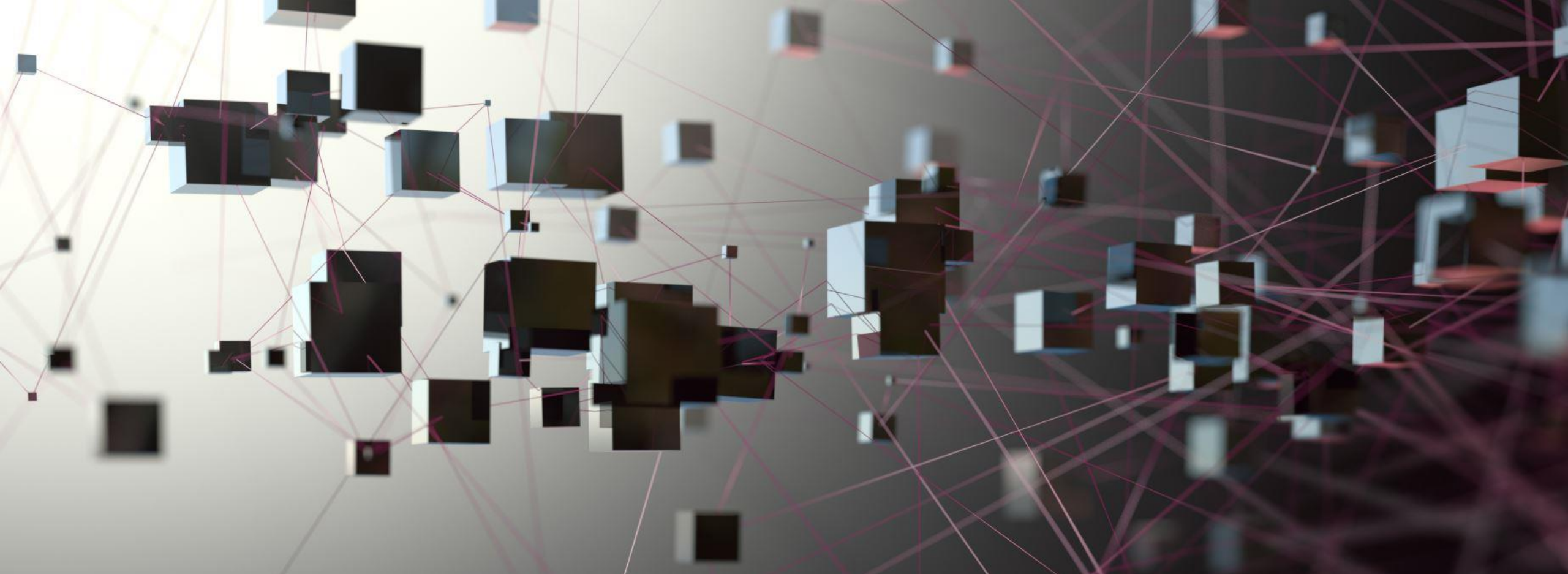
Callback de la interrupción por completar la conversión

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
```

# Comparación de métodos

---

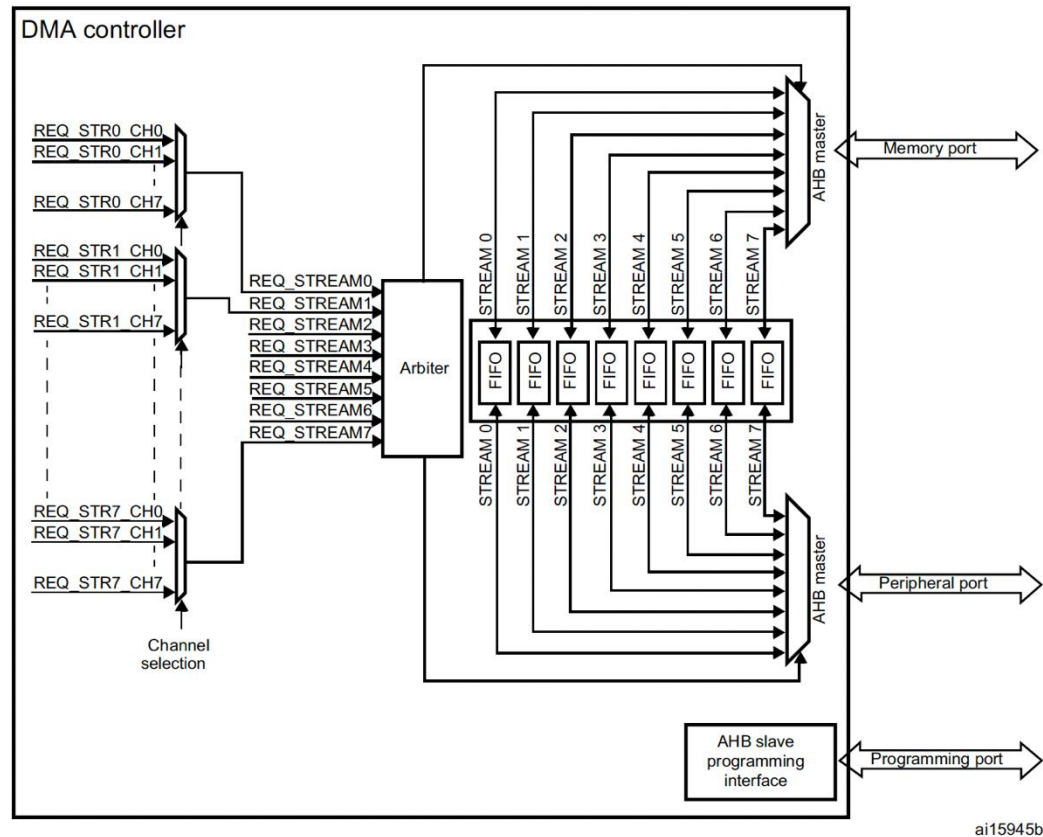
| Método    | CPU bloqueada | Uso típico        |
|-----------|---------------|-------------------|
| Polling   | Sí            | Pruebas           |
| Interrupt | No            | Eventos           |
| DMA       | No            | Muestreo continuo |



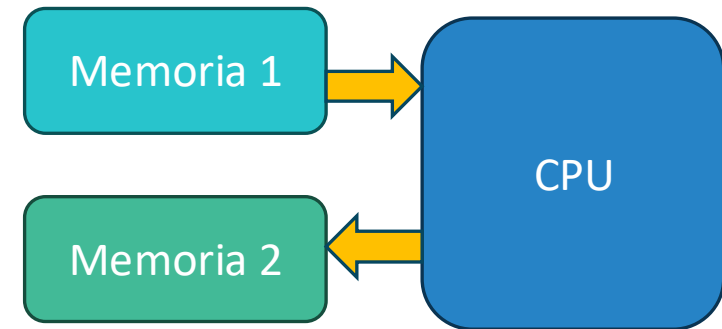
# DMA STM32

# DMA

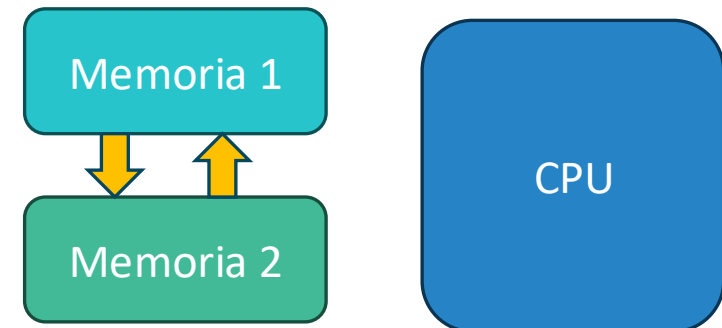
Ejemplo: DMA permite que el ADC funcione como un sistema autónomo.



Sin DMA

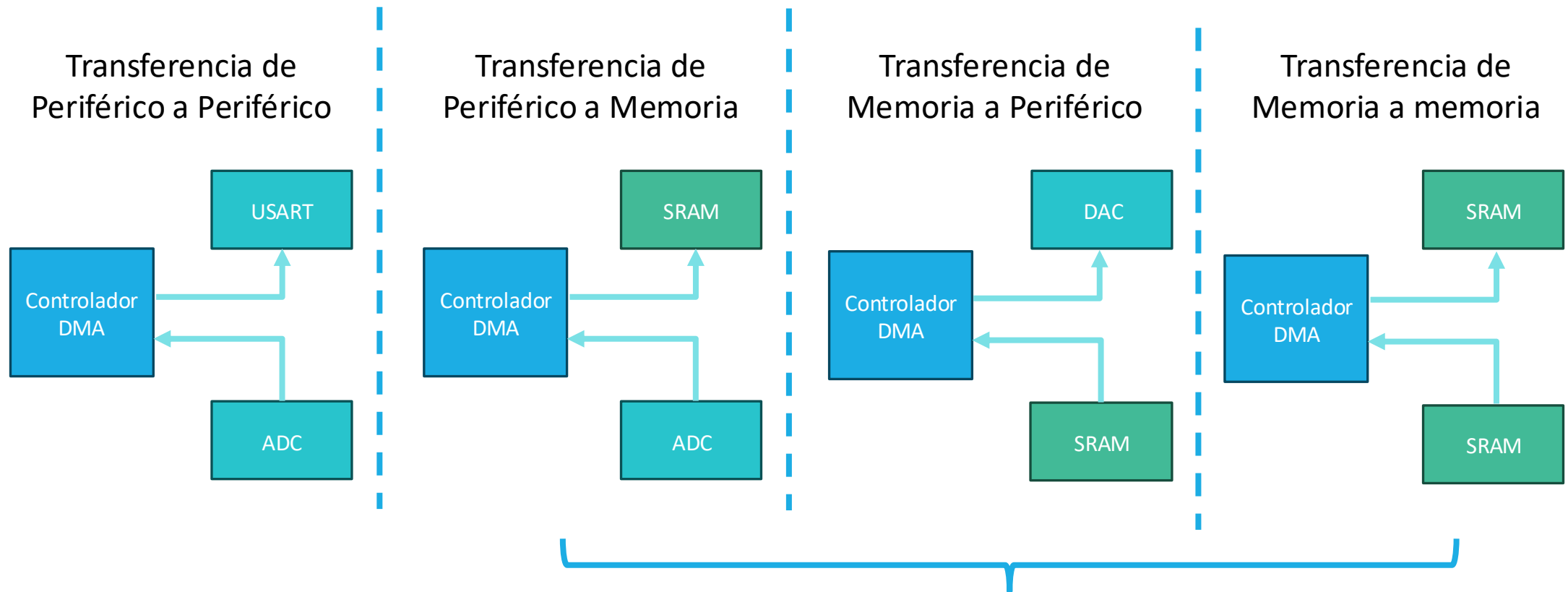


Con DMA





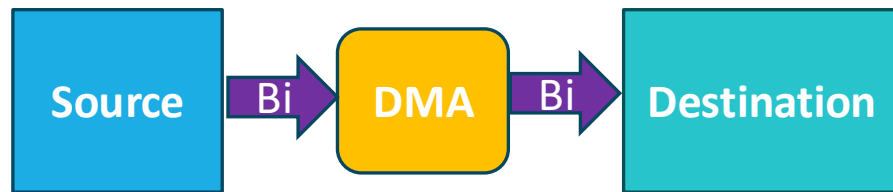
# DMA Tipos de transferencias



# DMA configuraciones

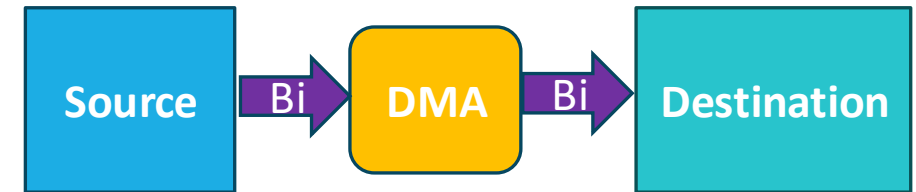
Puede configurarse la longitud del dato

Si los tamaños de datos de origen y destino no coinciden, el controlador DMA se encarga de realizar las organizaciones y adaptaciones de los datos.



| Address | Data[7:0]      |
|---------|----------------|
| 0x00    | B <sub>0</sub> |
| 0x01    | B <sub>1</sub> |
| 0x02    | B <sub>2</sub> |
| 0x03    | B <sub>3</sub> |

| Address | Data[31:0]          |
|---------|---------------------|
| 0x0000  | 0x000B <sub>0</sub> |
| 0x0004  | 0x000B <sub>1</sub> |
| 0x0008  | 0x000B <sub>2</sub> |
| 0x000C  | 0x000B <sub>3</sub> |



| Address | Data[31:0]                                                  |
|---------|-------------------------------------------------------------|
| 0x0000  | B <sub>3</sub> B <sub>2</sub> B <sub>1</sub> B <sub>0</sub> |
| 0x0004  | B <sub>7</sub> B <sub>6</sub> B <sub>5</sub> B <sub>4</sub> |
| 0x0008  | B <sub>B</sub> B <sub>A</sub> B <sub>9</sub> B <sub>8</sub> |
| 0x000C  | B <sub>F</sub> B <sub>E</sub> B <sub>D</sub> B <sub>C</sub> |

| Address | Data[15:0]                    |
|---------|-------------------------------|
| 0x0000  | B <sub>1</sub> B <sub>0</sub> |
| 0x0004  | B <sub>5</sub> B <sub>4</sub> |
| 0x0008  | B <sub>9</sub> B <sub>8</sub> |
| 0x000C  | B <sub>D</sub> B <sub>C</sub> |

# DMA - Configuraciones

---

Las direcciones de memoria desde y hacia donde se realizan las transferencias puede ser incrementadas o no incrementadas automáticamente

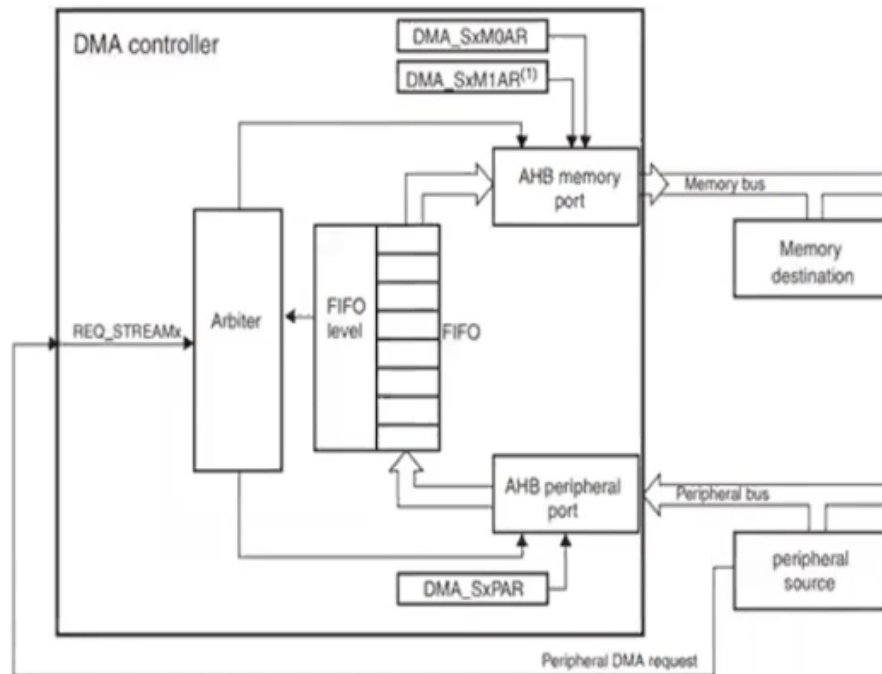
La cantidad de datos a transferir es configurable

Existen dos modos de transferencia de datos:

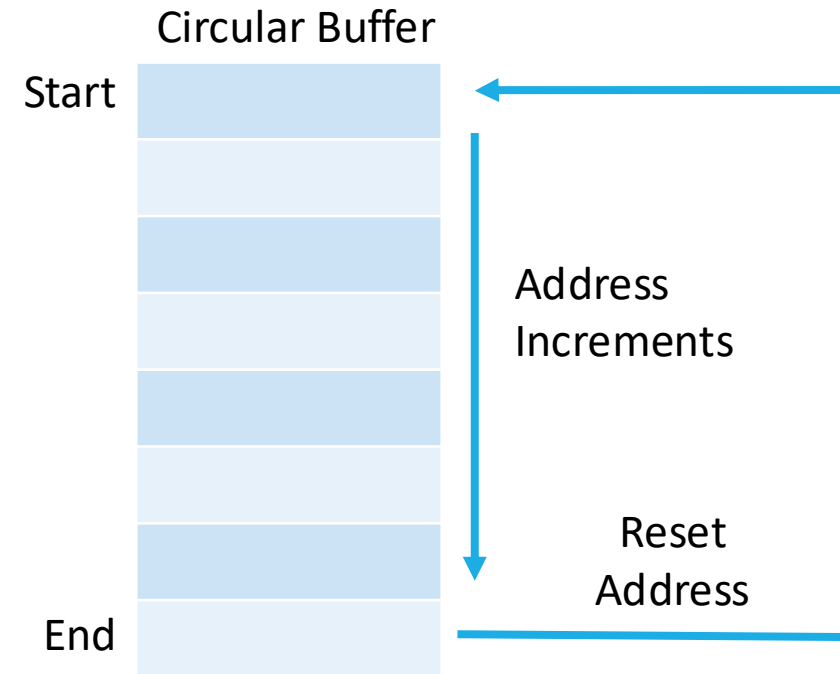
- **Normal:** se transfiere la cantidad de datos configurados. Una vez que el último dato es transmitido, el flujo se detiene y debe ser iniciado nuevamente en forma manual.
- **Circular:** permite el uso de la memoria como buffer de datos obtenidos en forma continua. Una vez finalizada la transmisión, se reinicia automáticamente con la misma cantidad de datos y sobre el / los bloques de memoria utilizados anteriormente.

# DMA - Configuraciones

## MODO NORMAL



## MODO CIRCULAR



# DMA – STM32F446RE

Posee dos módulos DMA (DMA1 y DMA2)

Con 8 canales independientes de transferencia cada uno

Canales configurables para transferencias

- Memoria - Memoria
- Periférico – Memoria

Prioridades de canales configurables por software

- Baja
- Media
- Alta
- Muy alta

Capacidad de transferencia de hasta 65536 datos

3 indicaciones de estado con su correspondiente interrupción:

- Media transferencia completa
- Transferencia completa
- Error en transferencia



| Peripheral requests | Stream 0            | Stream 1              | Stream 2            | Stream 3              | Stream 4              | Stream 5    | Stream 6             | Stream 7            |
|---------------------|---------------------|-----------------------|---------------------|-----------------------|-----------------------|-------------|----------------------|---------------------|
| Channel 0           | SPI3_RX             | -                     | SPI3_RX             | SPI2_RX               | SPI2_TX               | SPI3_TX     | -                    | SPI3_TX             |
| Channel 1           | I2C1_RX             | -                     | TIM7_UP             |                       | TIM7_UP               | I2C1_RX     | I2C1_TX              | I2C1_TX             |
| Channel 2           | TIM4_CH1            | -                     | I2S3_EXT_RX         | TIM4_CH2              | I2S2_EXT_TX           | I2S3_EXT_TX | TIM4_UP              | TIM4_CH3            |
| Channel 3           | I2S3_EXT_RX         | TIM2_UP<br>TIM2_CH3   | I2C3_RX             | I2S2_EXT_RX           | I2C3_TX               | TIM2_CH1    | TIM2_CH2<br>TIM2_CH4 | TIM2_UP<br>TIM2_CH4 |
| Channel 4           | UART5_RX            | USART3_RX             | UART4_RX            | USART3_TX             | UART4_TX              | USART2_RX   | USART2_TX            | UART5_TX            |
| Channel 5           | UART8_TX            | UART7_TX              | TIM3_CH4<br>TIM3_UP | UART7_RX              | TIM3_CH1<br>TIM3_TRIG | TIM3_CH2    | UART8_RX             | TIM3_CH3            |
| Channel 6           | TIM5_CH3<br>TIM5_UP | TIM5_CH4<br>TIM5_TRIG | TIM5_CH1            | TIM5_CH4<br>TIM5_TRIG | TIM5_CH2              | -           | TIM5_UP              | -                   |
| Channel 7           | -                   | TIM6_UP               | I2C2_RX             | I2C2_RX               | USART3_TX             | DAC1        | DAC2                 | I2C2_TX             |

The DMA can be used with the main peripherals:

- SPI and I<sup>2</sup>S
- I<sup>2</sup>C
- USART
- General-purpose, basic and advanced-control timers TIMx
- DAC
- SDIO
- Camera interface (DCMI)
- ADC
- SAI1/SAI2
- SPDIF Receiver (SPDIFRx)
- QuadSPI

# DMA – STM32F446RE

# Librería HAL y DMA

---

Las transferencias de datos son contrables con HAL

Si un periférico está involucrado, las transferencias se inician con:

***HAL\_XXX\_Start\_DMA();***

Los eventos de DMA pueden generar interrupciones:

- Media transferencia: ***HAL\_XXX\_HalfCpltCallback()***
- Transferencia completa: ***HAL\_XXX\_CpltCallback()***
- Error: ***HAL\_XXX\_ErrorCallback()***

# DMA Memoria - Memoria

---

Requiere establecer:

- Prioridad
- Modo de transferencia: normal o circular (**memoria a memoria solo se utiliza normal**)
- Tamaño de los datos (byte, half-word, o word)

Para realizar una transferencia de datos desde una sección a otra, se puede utilizar la instrucción

***HAL\_DMA\_Start(DMA\_HandleTypeDef \*hdma, uint32\_t SrcAddress, uint32\_t DstAddress, uint32 DataLength);***

Esta función requiere como parámetros el acceso al periférico, la dirección de origen de los datos, la dirección de destino de los datos y el número de datos a transferir.

Su ejecución es bloqueante



# DMA Memoria - Memoria

---

Para realizar una transferencia de datos desde una sección a otra, se puede utilizar la instrucción

```
HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength);
```

Esta función requiere como parámetros el acceso al periférico, la dirección de origen de los datos, la dirección de destino de los datos y el número de datos a transferir.

Su ejecución NO es bloqueante

No existe Callback predefinido para DMA Memoria – Memoria

El usuario puede definir una rutina que luego declarará como rutina de interrupción

Debe cumplir con:

```
void Nombre_Función(DMA_HandleTypeDef * _hdma);
```

```
HAL_DMA_registerCallback(DMA_HandleTypeDef *hdma, HAL_DMA_CallbackIDTypeDef CallbackID, void (*pCallback) (DMA_HandleTypeDef * _hdma));
```

# DMA Memoria - Memoria

---

Es posible definir rutinas para diferentes eventos de interrupción:

- ***HAL\_DMA\_XFER\_CPLT\_CB\_ID:** transferencia de datos finalizada exitosamente*
- ***HAL\_DMA\_XFER\_HALFCPLT\_CB\_ID:** mitad de la transferencia realizada*
- ***HAL\_DMA\_XFER\_ERROR\_CB\_ID:** error en la transferencia*
- ***HAL\_DMA\_XFER\_ABORT\_CB\_ID:** transferencia abortada por el usuario*
- ***HAL\_DMA\_XFER\_ALL\_CB\_ID:** cualquiera de los eventos anteriores*

# Bibliografía

---

Mastering STM32 segunda edición(Carmine Noviello, 2024)

<https://www.st.com/en/evaluation-tools/nucleo-f446re.html>

<https://www.st.com/en/microcontrollers-microprocessors/stm32f446re.html>

<https://controllerstech.com/stm32-adc-series/>